

Bronze → Silver Cleaning Report

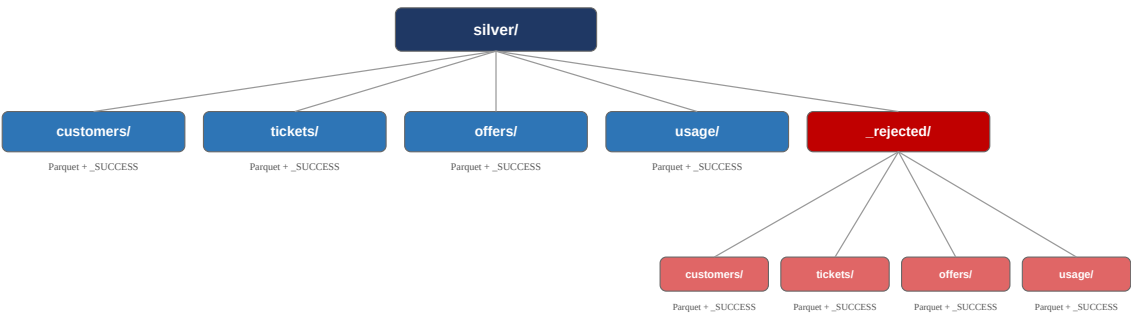
Pipeline: Ingestion (Bronze) → Standardization → Deduplication → Validation → Business-key dedupe → Silver. Executed via a single staged PySpark script (clean_all_bronze_to_silver.py) covering four sources: Customers, Customer Support Tickets, Offers, Usage.

Overall Summary

Source	Source Rows	Clean Rows	Rejected Rows	Unaccounted	Rejection Rate
Customers	1023	684	339	0	33.14%
Customer Support Tickets	1528	1316	212	0	13.87%
Offers	1535	1417	118	0	7.69%
Usage	3087	2812	275	0	8.91%

"Unaccounted" = Source rows – (Clean + Rejected). A non-zero value indicates rows lost during processing (e.g. due to null-unsafe filter conditions); all four sources reconcile to zero.

Silver Layer Folder Structure



"tickets/" above refers to customer_support_tickets/. Each leaf folder holds coalesced Parquet part-files plus a _SUCCESS marker. _rejected/ mirrors the same four sources, holding every row that failed validation, deduplication, or the foreign-key check, tagged with a rejection_reason column.

Validation Rules Applied

Customers

Rule	Detail
Primary key	CustomerId not null and unique
Age	Between 18 and 90
CreditScore	Between 300 and 850
Geography	Not null, in {France, Spain, Germany}
Gender	Not null, in {Male, Female}
Tenure	Not negative and less than Age
Balance / EstimatedSalary	Not null and not negative
NumOfProducts / tenure_months	Not negative

Rule	Detail
HasCrCard / IsActiveMember / Exited	Not null, must be 0 or 1
date_opened	Not null and not a future date

Customer Support Tickets

Rule	Detail
Primary key	ticket_id not null and unique
Foreign key	customer_id must exist in the Customers identity reference
issue_type	Not null, in known category set
severity	Not null, in {low, medium, high, critical}
resolution_time_hrs	Not null, between 0 and 720 hours (30 days)
created_at	Not null and not a future date

Offers

Rule	Detail
Primary key	offer_id not null and unique
offer_type	Not null, in known category set
accepted	Not null, normalized to 0/1 (accepts Yes/No/Y/N/true/false variants)
date_offered	Not null and not a future date

Usage

Rule	Detail
Primary key	usage_log_id not null and unique
Foreign key	customer_id must exist in the Customers identity reference
usage_month	Not null, normalized to YYYY-MM, not a future month
product_type	Not null
monthly_balance	Not null and not negative
num_products	Not null and not negative

Rejection Breakdown by Source

Customers — 339 rejected (33.14%)

Reason	Count
invalid_exited	87
invalid_gender	49
invalid_credit_score	47
invalid_balance	36
invalid_has_cr_card	35
invalid_estimated_salary	23
invalid_geography	19
exact_duplicate	15
invalid_age	14
invalid_tenure	8
duplicate_customer_id	6

Customer Support Tickets — 212 rejected (13.87%)

Reason	Count
invalid_resolution_time	97
invalid_issue_type	43
exact_duplicate	29
invalid_severity	28
orphan_customer_id	15

Offers — 118 rejected (7.69%)

Reason	Count
invalid_accepted	58
exact_duplicate	30
invalid_offer_type	30

Usage — 275 rejected (8.91%)

Reason	Count
invalid_monthly_balance	186
exact_duplicate	60

Reason	Count
missing_product_type	17
invalid_num_products	12

Note: Customers shows the highest rejection rate (33.14%) because it carries the most validation rules (11 checks across identity, demographic, and account fields) compared to 5-6 checks on the other sources. This is expected given the richer schema, not a data-quality anomaly.

Methodology & Error Handling

Staged processing: Each source moves through four checkpointed stages (Standardize → Deduplicate → Validate → Finalize), with intermediate results written to and re-read from Parquet between stages. This keeps each stage's Spark logical plan small and avoids driver out-of-memory errors caused by long chained transformations.

Exact-duplicate detection: Implemented via a SHA-256 hash of all columns per row, partitioned in a single column, rather than a window function partitioned across every column individually — significantly lighter on the Catalyst optimizer.

Null-safe validation: All categorical checks (isin) are explicitly guarded with isNotNull() before the membership test. Spark's three-valued logic means isin() on a NULL value returns NULL rather than False, which — left unguarded — causes rows to silently disappear from both the valid and rejected outputs. This was identified and fixed across all four sources during development (see Unaccounted rows = 0 above).

Foreign key checks: Tickets and Usage validate customer_id against a dedicated identity reference table (distinct, non-null CustomerId values captured immediately after Customers standardization) rather than against the fully-validated Customers Silver table. This avoids incorrectly flagging tickets/usage records as orphaned merely because the referenced customer failed an unrelated validation rule (e.g. Gender formatting). The reference is broadcast-joined given its small size, avoiding a shuffle join.

Error handling: The pipeline runs all four sources sequentially inside a try/except per source. Customers must succeed first since Tickets and Usage depend on it for the foreign key check; a Customers failure halts the pipeline. A failure in any other source is caught, logged, and does not block the remaining sources. A final pipeline summary reports success/failure per source.